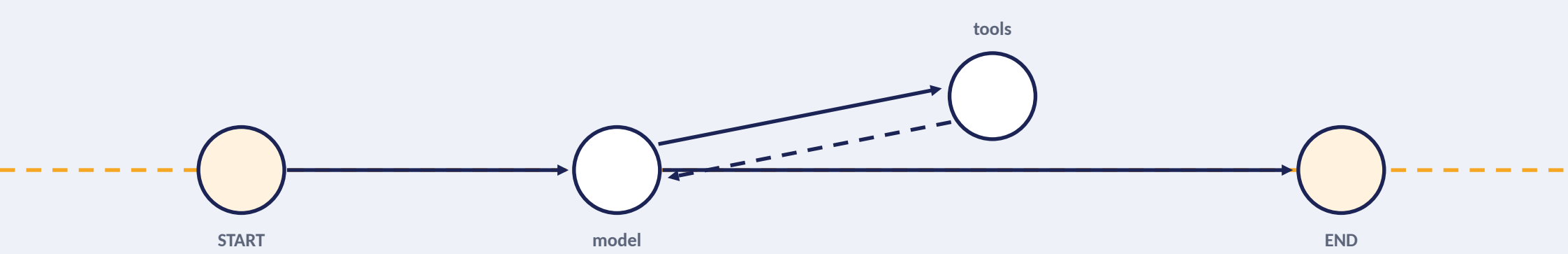
LANGGRAPH · CHAPTER 7 · FLASHCARDS

Agents and the ReAct Loop

One back edge, many laps

12 code cards on the agent loop, built four ways and capped



The LLM node and the router

The model decides on every lap; the router only reads that decision

- llm_call returns one new message: either tool_calls or a plain answer.
- The system prompt is added at call time, so it is never stored in the history.

PYTHON

```python
from langchain_core.messages import SystemMessage
from langgraph.graph import END, MessagesState


def llm_call(state: MessagesState):
    """The model decides: request a tool, or answer."""
    return {"messages": [llm_with_tools.invoke(
        [SystemMessage("You are a helpful assistant tasked with performing "
                       "arithmetic on a set of inputs.")]
        + state["messages"]          # the whole run so far, every lap
    )]}


def should_continue(state: MessagesState) -> str:
    """Route to the tools if the model requested any, otherwise end."""
    last_message = state["messages"][-1]
    if getattr(last_message, "tool_calls", None):    # no judgment of its own
        return "tool_node"
    return END
```

One edge makes it an agent

Point the tool node back at the model and the model can act again on what it saw

- Chapter 6's tools led forward to an answer node; here tool_node leads back to llm_call.
- The edge list fixes no lap count: the model decides how many laps to run.

PYTHON

```python
from langgraph.graph import START, StateGraph

builder = StateGraph(MessagesState)
builder.add_node("llm_call", llm_call)
builder.add_node("tool_node", tool_node)          # hand-written, Listing 6.6

builder.add_edge(START, "llm_call")
builder.add_conditional_edges("llm_call", should_continue, ["tool_node", END])
builder.add_edge("tool_node", "llm_call")         # <-- THE BACK EDGE
agent = builder.compile()

result = agent.invoke(
    {"messages": [HumanMessage("Add 3 and 4. Then multiply the result by 2.")]})
for m in result["messages"]:
    m.pretty_print()     # Human, AI(add), Tool 7, AI(multiply), Tool 14, answer
```

What connects one lap to the next

Each lap is reason, act, observe; add_messages keeps every step in the list

- The model is stateless: it can ask for multiply(7, 2) only because the 7 is in messages.
- Own state class? Declare messages: Annotated[list, add_messages] or subclass MessagesState.

TEXT

```
Question: "Add 3 and 4. Then multiply the result by 2."

lap 1   llm_call  -> AI: add(3, 4)         reason + act   (router: tool_calls)
        tool_node -> Tool("7")             observe        (back edge -> llm_call)
lap 2   llm_call  -> AI: multiply(7, 2)    reads the 7 it just observed
        tool_node -> Tool("14")            observe        (back edge -> llm_call)
answer  llm_call  -> AI: "... is 14."      no tool_calls  (router: END)

messages with add_messages (each node appends its one new message):
  [Human] -> [Human, AI(add)] -> [Human, AI(add), Tool("7")] -> ...

messages as a plain list (each write replaces the list):
  [Human] -> [AI(add)] -> [Tool("7")]      lap 2 gets a bare 7, no question

(plain-list overwrite checked offline on langgraph 1.2.6)
```

ToolNode and tools_condition

Swap the hand-written tool node and router for prebuilt ones; the back edge stays

- ToolNode runs every requested call and appends a ToolMessage with the matching id.
- tools_condition returns "tools" when the last message has tool_calls, otherwise END.

PYTHON

```python
from langgraph.prebuilt import ToolNode, tools_condition

builder = StateGraph(MessagesState)
builder.add_node("llm", call_llm)
builder.add_node("tools", ToolNode(tools))   # the name tools_condition routes to
builder.add_edge(START, "llm")
builder.add_conditional_edges("llm", tools_condition)     # "tools" or END
builder.add_edge("tools", "llm")                           # the back edge
agent = builder.compile()

# What the router returns (checked offline):
tools_condition({"messages": [AIMessage("", tool_calls=[call])]})   # 'tools'
tools_condition({"messages": [AIMessage("Hi there")]})              # '__end__'
```

LEVEL 3 · COMMAND, NO MODEL

Update and route in one return

A node that returns Command(update=..., goto=...) needs no conditional edge

- The Command[Literal[...]] annotation lets compile() check targets and draw the edges.
- billing passing a refund to refunds is a handoff: the same move agents make later.

PYTHON

```python
def triage(state: State) -> Command[Literal["billing", "tech"]]:
    text = state["request"].lower()
    desk = ("billing" if any(w in text for w in ("invoice", "refund", "charge"))
            else "tech")
    return Command(update={"desk": desk, "path": ["triage"]},  # the update...
                   goto=desk)                                  # ...and the next node


def billing(state: State) -> Command[Literal["refunds", "__end__"]]:
    if "refund" in state["request"].lower():                   # a handoff
        return Command(update={"desk": "refunds",
                               "path": state["path"] + ["billing"]},
                       goto="refunds")
    return Command(update={"path": state["path"] + ["billing"],
                           "reply": "Billing: we have corrected your invoice."},
                   goto=END)

# python "6-Agents/06_command_basics.py"   (no API key needed)
# 'My invoice is wrong'        -> triage -> billing -> END
# 'Please refund my order'     -> triage -> billing -> refunds -> END
# 'The app crashes on login'   -> triage -> tech -> END
```

LEVEL 3 · COMMAND IN THE LOOP

The LLM node routes itself

The model node appends its reply and names the next hop in the same Command

- No router function and no add_conditional_edges; the back edge tools -> llm stays.
- Conditional edges keep routing visible; use Command when a node owns both jobs.

PYTHON

```python
def call_model(state: MessagesState) -> Command[Literal["tools", "__end__"]]:
    response = model.invoke(
        [SystemMessage("You are an arithmetic assistant. "
                       "Use the provided tools for calculations."),
         *state["messages"]])
    next_node = "tools" if response.tool_calls else END
    return Command(
        update={"messages": [response]},    # the state update...
        goto=next_node,                     # ...and the next hop, together
    )


builder = StateGraph(MessagesState)
builder.add_node("llm", call_model)
builder.add_node("tools", ToolNode(tools))
builder.add_edge(START, "llm")
builder.add_edge("tools", "llm")       # the back edge stays; no conditional edge
graph = builder.compile()
```

The whole loop in one call

create_agent builds the model node, tools node, router and back edge for you

- Its tools node runs every call the model asks for: no code of yours sits in between.
- It returns an ordinary compiled graph you can invoke, stream, checkpoint and draw.

PYTHON

```python
from langchain.agents import create_agent

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[add, multiply],          # these RUN automatically, on the model's say-so
    system_prompt="You are an arithmetic assistant. "
                  "Always use the provided tools for calculations.",
)
question = HumanMessage("Add 3 and 4, then multiply the result by 2.")
result = agent.invoke({"messages": [question]})

g = agent.get_graph()               # the loop you built by hand
print(list(g.nodes))
# ['__start__', 'model', 'tools', '__end__']
print([(e.source, e.target) for e in g.edges])
# [('__start__', 'model'), ('model', '__end__'),
#  ('model', 'tools'), ('tools', 'model')]          <- the back edge
```

The result is the message list

invoke returns a dict whose messages key holds the whole run, answer last

- messages[-1] is the answer only if the run ended normally; after a limit it is a status.
- Need extra keys? Extend AgentState: messages is inherited, declare only what is new.

PYTHON

```python
print(type(result).__name__, list(result.keys()))   # dict ['messages']

answer = result["messages"][-1].content           # the final answer
for m in result["messages"]:
    m.pretty_print()                              # the whole run, formatted
print([(m.name, m.content) for m in result["messages"]
       if isinstance(m, ToolMessage)])            # which tools ran
# [('add', '7'), ('multiply', '4'), ('multiply', '14')]   <- a wasted guess


from langchain.agents import AgentState, create_agent

class SupportState(AgentState):      # messages is inherited
    user_id: str

agent = create_agent(model="openai:gpt-4o-mini", tools=tools,
                     state_schema=SupportState)
result = agent.invoke({"messages": [HumanMessage("Where is my order?")],
                       "user_id": "u42"})
print(list(result.keys()))           # ['messages', 'user_id']
```

The model decides when to search

Give search_docs to the loop and the model picks whether, what and how often to search

- One graph, many paths: no search for general knowledge, two searches for a two-part question.
- ToolNode returns bad arguments to the model but re-raises ConnectionError, so the retry fires.

PYTHON

```python
def build_agent(vectorstore):
    tools = [make_search_tool(vectorstore)]           # Chapter 6's search_docs
    llm_with_tools = ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(tools)

    def call_llm(state: AgentState) -> dict:
        return {"messages": [llm_with_tools.invoke(
            [SystemMessage(SYSTEM_PROMPT)] + state["messages"])]}

    builder = StateGraph(AgentState)
    builder.add_node("llm", call_llm)
    builder.add_node("tools", ToolNode(tools),       # a ConnectionError re-runs it
                     retry_policy=RetryPolicy(max_attempts=3,
                                              retry_on=ConnectionError))
    builder.add_edge(START, "llm")
    builder.add_conditional_edges("llm", tools_condition)
    builder.add_edge("tools", "llm")                  # the back edge
    return builder.compile()

agent = build_agent(vectorstore)
agent.invoke({"messages": [HumanMessage(question)]},
             config={"recursion_limit": 10})   # 10 steps = about 5 laps, then error
```

An iteration cap in state

The model's answer is a stopping signal; a counter in state is the guarantee

- The router checks the cap first, so it ends the run whatever the model wants.
- When it fires, the last message is an unanswered tool request: route to a reply node.

PYTHON

```python
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
    iteration_count: int


def call_llm(state: AgentState) -> dict:
    iteration = state["iteration_count"] + 1
    print(f"Iteration {iteration}")
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response], "iteration_count": iteration}


def should_use_tools(state: AgentState) -> str:
    if state["iteration_count"] >= 5:                # the guarantee
        print("Iteration cap reached — stopping.")
        return END
    last = state["messages"][-1]
    return "tools" if getattr(last, "tool_calls", None) else END   # the signal

# Iteration 1 ... Iteration 5, then "Iteration cap reached — stopping."
# 5 model calls, 4 tool runs (checked offline with a scripted model)
```

Call limits as middleware

With create_agent you pass limit middleware instead of writing a counter

- Both runs end cleanly and the last message says why; exit_behavior="error" raises instead.
- The tool limit defaults to "continue": a stubborn model then runs to the recursion limit.

PYTHON

```python
from langchain.agents.middleware import (ModelCallLimitMiddleware,
                                         ToolCallLimitMiddleware)

@tool
def check_status() -> str:
    """Check whether the nightly backup job has finished."""
    return "Still pending. Check again."     # never finishes: a runaway on purpose

def run(label, middleware):
    agent = create_agent(model="openai:gpt-4o-mini", tools=[check_status],
                         middleware=[middleware])   # the only difference
    result = agent.invoke({"messages": [HumanMessage(REQUEST)]})
    print(f"{label}: {result['messages'][-1].content}")   # a status, not an answer

run("Model-call limit (3)", ModelCallLimitMiddleware(run_limit=3))
# Model call limits exceeded: run limit (3/3)        <- check_status ran 3 times
run("Tool-call limit (2)",
    ToolCallLimitMiddleware(tool_name="check_status", run_limit=2,
                            exit_behavior="end"))
# 'check_status' tool call limit reached: run limit exceeded (3/2 calls).
```

Four quiet loop mistakes

None of these raises at compile time; the run just goes somewhere you did not plan

- A node returning Command does its own routing: a conditional edge on it adds a second hop.
- The Literal is checked at compile(), not at run time: a goto to a missing node just stops.

PYTHON

```python
# 1. No back edge: one round of tools, then the run ends (Chapter 6 again)
builder.add_edge("tool_node", END)              # should go back to "llm_call"

# 2. Command AND a conditional edge on one node: BOTH targets run
def a(state) -> Command[Literal["b"]]:
    return Command(update={"path": ["a"]}, goto="b")
builder.add_conditional_edges("a", lambda s: "c", ["c"])
# {'path': ['a', 'b', 'c']}

# 3. No Command[Literal[...]] annotation: it runs, but the drawing is wrong
# edges: ('__start__', 'a'), ('a', '__end__')     <- no a -> b at all

# 4. goto a name that is not a node: no exception, the run stops there
return Command(goto="zzz")     # only a log line: "wrote to unknown channel ..."

# Checked on langgraph 1.2.6
```